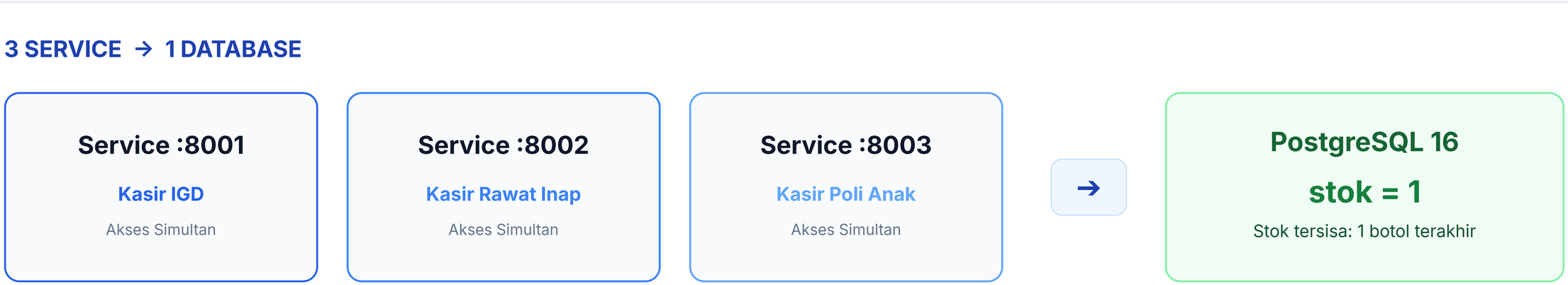
SISTEM TERDISTRIBUSI • FASILKOM UNEJ

Konsistensi Data & Race Condition

Studi Kasus: Multi-Instance Layanan Apotek dengan PostgreSQL Terpusat

Analisis Komparasi: Naive, Atomic Operation, Database Transaction, dan Optimistic Locking

Teknologi: Laravel (PHP 8.3) • PostgreSQL 16 • Concurrency Engine Simulator



Kelompok 2: Ridho Rizky P. • Firsandi Andraw F. • Ahmad Rafif Nandra U.

ARSITEKTUR SISTEM

Model Skalabilitas: Multi Order Service & Single Database

1 Multi-Client Concurrency

- Kasir IGD
- Kasir Rawat Inap
- Kasir Poli Anak

3 Kasir Rumah Sakit (IGD, Rawat Inap, Poli Anak) melakukan transaksi tebus resep secara simultan di milidetik yang sama saat jam sibuk operasional rumah sakit.

2 Multi Service Instances

- Order Service :8001
- Order Service :8002
- Order Service :8003

3 Instance Apotek Order Service berjalan mandiri di port 8001, 8002, dan 8003 untuk load balancing, isolasi beban, dan ketersediaan tinggi (High Availability).

3 Single Shared Database

- PostgreSQL 16
- obats (stok obat)
- pesanan_obats

Ketiga service terhubung ke satu database PostgreSQL terpusat yang menyimpan tabel obats (stok obat) dan pesanan_obats sebagai Single Source of Truth.

POLA MASALAH - SOLUSI

Mengapa Data Tidak Konsisten Meski Database Hanya Satu?

MASALAH NYATA: STOK OBAT KRITIS TERSISA 1

Kasus: Obat 'Amoxicillin Sirup' hanya tersisa 1 botol terakhir.

Serentak: 3 Kasir (IGD, Rawat Inap, Poli) menekan tombol konfirmasi tebus resep.

Race Condition: Ketiga service membaca stok = 1 pada saat bersamaan.

Overselling: 3 botol dinyatakan berhasil terjual padahal fisik obat hanya ada 1 botol!

Dampak Medis: Pasien yang resepnya disetujui tidak mendapatkan obat karena stok fisik kosong di lemari farmasi.

1 botol tersisa → 3 botol terjual

OVERSELLING FATAL: ANCAMAN KESELAMATAN PASIEN

ARAH SOLUSI: KONSISTENSI DI LEVEL DATABASE

Akar Masalah: Aplikasi mempercayai data lokal di RAM yang sudah 'usang' (stale read).

Prinsip Solusi: Memindahkan validasi stok dari memori aplikasi PHP ke mesin PostgreSQL.

3 Pendekatan Solusi Teruji:

1. Operasi Atomic

UPDATE ... WHERE stok >= 1 langsung di level SQL.

2. Database Transaction

SELECT FOR UPDATE: kunci baris data hingga commit/rollback.

3. Optimistic Locking

Lacak nomor versi record (kolom versi) tanpa mengunci pembaca.

MASA LALU (THE NAIVE ERA)

Kegagalan Pola 'Read-Modify-Write' Saat Layanan Di-Scale

No	Pelaku / Instance	Aksi Pemrosesan	Database (Stok)	Evaluasi Status
1	**Service #1 (Port 8001)**	Membaca stok obat dari DB ➔ Hasil: 1	**Stok: 1**	**Valid**
2	**Service #2 (Port 8002)**	Membaca stok obat dari DB ➔ Hasil: 1	**Stok: 1**	**Valid (Stale Read)**
3	**Service #3 (Port 8003)**	Membaca stok obat dari DB ➔ Hasil: 1	**Stok: 1**	**Valid (Stale Read)**
4	**Service #1**	Perhitungan di RAM: 1 - 1 = 0 ➔ Simpan Stok = 0	**Stok: 0**	**Order #1 Sukses**
5	**Service #2**	Perhitungan di RAM: 1 - 1 = 0 ➔ Simpan Stok = 0	**Stok: 0**	**Order #2 Sukses (OVERSELL!)**
6	**Service #3**	Perhitungan di RAM: 1 - 1 = 0 ➔ Simpan Stok = 0	**Stok: 0**	**Order #3 Sukses (OVERSELL!)**

Langkah 1-3: Ketiga service membaca stok = 1 sebelum ada yang menulis.
Langkah 5-6: Hasil hitung dari data usang tetap disimpan, sehingga terjadi overselling fatal!

4 • RULE OF THREE

RULE OF THREE

Tiga Pilar Menjaga Konsistensi Data Sistem Terdistribusi

PILAR 1

Atomic Operation

EKSEKUSI TUNGGAL DI SQL

Menyerahkan pengurangan stok langsung ke mesin PostgreSQL. Tidak ada data yang dibaca ke memori PHP terlebih dahulu.

SQL Engine Execution

```
UPDATE obats
SET stok = stok - 1
WHERE id = 1 AND stok >= 1;
```

PILAR 2

Database Transaction

PESSIMISTIC LOCKING (ACID)

Mengunci baris obat (SELECT FOR UPDATE) sehingga service lain harus antre menunggu hingga transaksi commit atau rollback.

SQL Engine Execution

```
BEGIN TRANSACTION;
SELECT ... FOR UPDATE;
COMMIT / ROLLBACK;
```

PILAR 3

Optimistic Locking

VERSION TRACKING

Bebas antrean kunci. Setiap pembaruan memeriksa nomor versi data (kolom versi). Jika versi berubah, transaksi otomatis dibatalkan.

SQL Engine Execution

```
UPDATE obats
SET stok = 0, versi = 6
WHERE id = 1 AND versi = 5;
```

MASA SEKARANG: SOLUSI 1

Operasi Atomic: Mengandalkan Conditional Update di SQL

Mekanisme Kerja

Alih-alih membaca stok ke PHP, query langsung dijalankan:

```
UPDATE obats SET stok = stok - :qty
WHERE id = :id AND stok >= :qty;
```

PostgreSQL mengembalikan 'Affected Rows':

1 baris Stok berhasil dipotong! Lanjut simpan pesanan.

0 baris Stok tidak cukup! Batalkan dan kembalikan HTTP 400.

Keunggulan:

Sangat cepat, tanpa overhead transaksi panjang, dan non-blocking.

PERTANYAAN KRITIS

'Bagaimana jika stok berhasil dikurangi, namun pesanan gagal disimpan ke database?'

Kelemahan Operasi Atomic Mandiri:

Jika server crash, koneksi putus, atau insert tabel pesanan_obats error setelah update stok berhasil, maka stok obat di database berkurang permanen tanpa ada bukti resep tercatat di sistem!

Stok fisik di gudang dan catatan resep menjadi tidak sinkron.

Inilah alasan mengapa kita membutuhkan Solusi 2: Database Transaction (ACID All-or-Nothing)!

MASA SEKARANG: SOLUSI 2

Database Transaction (ACID): Menjamin 'All or Nothing'

Prinsip Transaksi

Tujuan: Jika salah satu tahapan gagal, perubahan yang terlanjur terjadi dapat dikembalikan ke kondisi semula (Rollback).

Alur Business Logic:

1. BEGIN TRANSACTION
2. SELECT * FROM obats WHERE id=1 FOR UPDATE (Lock baris)
3. Kurangi stok jika masih mencukupi
4. Simpan pesanan ke pesanan_obats
5. COMMIT TRANSACTION (Perubahan permanen)
6. Jika gagal di tengah jalan → ROLLBACK TRANSACTION

Keunggulan Integritas

Menjamin data tidak pernah korup. Kedua operasi (potong stok dan buat resep) berada dalam satu kesatuan atomik ACID.

Tidak ada skenario di mana stok berkurang tanpa resep tercipta, atau sebaliknya.

Konsekuensi Performa

- Mengunci baris data (lock) membuat service lain harus menunggu (blocking) hingga transaksi selesai.
- Dapat memicu bottleneck atau deadlock jika transaksi berjalan terlalu lama di bawah beban konkurensi ribuan request per detik.

MASA SEKARANG: SOLUSI 3

Optimistic Locking: Deteksi Konflik Berbasis Nomor Versi

Konsep Kolom Versi

• Asumsi Optimis: Konflik konkurensi tidak selalu terjadi, sehingga data tidak perlu dikunci di awal.

• Proses lain tetap bisa membaca data yang sama secara bebas (non-blocking reads).

• Menambahkan kolom 'versi' pada tabel obats (contoh: stok = 1, versi = 5).

• Setiap kali data diperbarui, nilai versi dinaikkan (versi = versi + 1).

• Saat menyimpan, sistem mengecek apakah versi di database masih sama dengan versi saat pertama dibaca.

stok = 1 | versi = 5

Timeline Eksekusi Versi

1. Service #1 membaca:
Stok = 1, Versi = 5

2. Service #2 membaca:
Stok = 1, Versi = 5

3. Service #1 update:
SET stok = 0, versi = 6 WHERE versi = 5
→ SUKSES • 1 row terupdate

4. Service #2 update:
SET stok = 0, versi = 6 WHERE versi = 5
→ DITOLAK • 0 row terupdate

Alasan Penolakan:

Versi di database sudah 6, bukan 5. Service #2 otomatis membatalkan pesanan (HTTP 409 Conflict) secara elegan tanpa lock!

BUKTI EKSPERIMEN NYATA

Hasil Uji Concurrency Paralel 3 Pembeli (Stok Awal = 1)

Skenario Uji	Service Instances	Respon Pembeli	Pesanan Masuk	Stok Akhir	Status Konsistensi
1. Naive (Tanpa Lock)	Port 8001, 8002, 8003	3x Berhasil (HTTP 200)	**3 Pesanan**	**0 (Bocor)**	**TIDAK KONSISTEN (OVERSELL)**
2. Atomic Operation	Port 8001, 8002, 8003	1x 200, 2x 400 Gagal	**1 Pesanan**	**0 (Akurat)**	**KONSISTEN (SQL Conditional)**
3. DB Transaction	Port 8001, 8002, 8003	1x 200, 2x 400 Gagal	**1 Pesanan**	**0 (Akurat)**	**KONSISTEN (ACID Rollback)**
4. Optimistic Locking	Port 8001, 8002, 8003	1x 200, 2x 409 Ditolak	**1 Pesanan**	**0 (Versi=6)**	**KONSISTEN (Version Check)**

3 pesanan masuk pada skenario Naive untuk stok 1:

Terjadi overselling fatal! Sistem mengonfirmasi 3 botol padahal barang fisik hanya 1.

1 pesanan masuk pada ketiga solusi teruji:

Stok akhir akurat dan konsisten 100%! Transaksi berlebih ditolak secara teratur.

9 • MASA DEPAN

MASA DEPAN (NEXT-GEN DISTRIBUTED SYSTEMS)

Tantangan Konkurensi Skala Lanjut (Database-per-Service)

REDLOCK ALGORITHM

Distributed Locks (Redis)

Ketika database dipecah menjadi beberapa cluster terpisah, lock database tunggal tidak cukup. Redis Distributed Lock mengunci resource lintas cluster di level in-memory.

ORKESTRASI ASINKRON

Saga Pattern & Event-Driven

Menggunakan event broker (Apache Kafka / RabbitMQ) untuk pemesanan obat, verifikasi stok, dan pembayaran dengan mekanisme compensating transactions.

PEMISAHAN BACA & TULIS

Eventual Consistency & CQRS

Command Query Responsibility Segregation (CQRS) memisahkan alur pembacaan stok dengan alur transaksi pemotongan agar throughput mencapai puluhan ribu RPS.

KESIMPULAN

Panduan Pemilihan Strategi Konsistensi Data

Pilih Operasi Atomic Jika...

Operasi hanya melibatkan 1 tabel tunggal, volume transaksi sangat tinggi, dan butuh performa tercepat tanpa overhead blocking.

Pilih DB Transaction Jika...

Operasi melibatkan beberapa tabel (stok + pesanan + audit log) yang wajib konsisten bersama ('All or Nothing') di 1 DBMS.

Pilih Optimistic Locking Jika...

Beban pembacaan jauh lebih tinggi daripada penulisan, latensi pengguna panjang (pengisian form), dan konflik jarang terjadi.

Demo Kode Praktikum & Simulator:

github.com/.../apotek-race-condition-pg

Q&A Session